# 2025 Big Data Analysis (DATA304)
# Final Project: Hierarchical Multi-Label Text Classification

Yejun Jung
School of Biomedical Engineering
College of Health Science
Korea University

## ABSTRACT

Hierarchical Multi-label Text Classification (HMTC) in a fully unlabeled setting is challenging due to the large, structured label space and the absence of gold supervision for learning fine-grained decision boundaries. This work proposes a weakly- and semi-supervised pipeline for product review classification with 531 hierarchical categories by leveraging only unlabeled corpora, taxonomy relations, class names, and class-related keywords. Document and label representations are constructed with pretrained BERT embeddings and further improved via domain-adaptive pretraining (MLM-based DAPT). Silver labels are generated by combining semantic and lexical similarities, followed by hierarchy-aware postprocessing to satisfy task constraints. A taxonomy-informed classifier is then trained using LabelGCN to propagate label information over the hierarchy and a GCN-enhanced matching head to score documents against refined label embeddings. To improve robustness under noisy supervision, the training procedure integrates self-training with pseudo-label generation and consistency regularization. Also it incorporates a limited number of LLM-assisted labels as high-precision anchors for uncertain samples. Experiments demonstrate that progressively adding these components yields substantial performance gains over the baseline with the best configuration achieved by combining SSL, DAPT embeddings, and selective LLM supervision.

## 1. INTRODUCTION

Hierarchical multi-label text classification (HMTC) arises in many real-world categorization problems where each document may belong to multiple categories that are organized in a taxonomy. Product review understanding is a representative example: a single review can describe several aspects of an item and the associated product categories often follow a hierarchy from coarse to fine-grained concepts. HMTC therefore requires not only predicting a set of relevant labels, but also capturing dependencies induced by the taxonomy such as correlations between parent and child categories.

This project considers a particularly challenging setting: classification must be performed without any manually labeled training data. The label space is large and structured, while each document is associated with a small set of labels. Under such constraints, directly training a supervised classifier is infeasible. In addition, predictions that ignore hierarchical relations may become inconsistent or unstable, especially when the decision boundary is learned from noisy signals. These characteristics motivate a framework that can bootstrap reliable supervision from unlabeled text, explicitly model label relations, and progressively refine the model using additional weak signals.

To address these challenges, we propose a weakly-supervised and semi-supervised pipeline built around three core ideas. First, we generate silver labels by aligning documents with label descriptions. We construct label texts from available class information such as class names and related keywords and compute document–label relevance using both semantic similarity in an embedding space and lexical matching. The resulting scores provide soft supervision that can be further regularized by simple hierarchy-aware heuristics. Second, we employ a hierarchy-aware classifier that integrates the taxonomy into the prediction model. Specifically, label embeddings are propagated over the label graph to encode structural dependencies and documents are scored against these structure-aware label representations. Third, we strengthen learning with semi-supervised strategies, which is a limited number of high-precision labels obtained via LLM-based annotation serve as anchors, while self-training expands the labeled set through pseudo-labels and consistency regularization encourages stable predictions under small input perturbations.

Overall, the proposed approach leverages the taxonomy, unlabeled corpora, and a limited annotation budget in a unified manner, turning a fully unlabeled HMTC problem into a tractable learning setup. Our experiments demonstrate that progressively adding these components from silver label bootstrapping, graph-based label modeling to LLM anchoring, and semi-supervised refinement consistently improves performance over the baseline.

**Contributions.** The main contributions of this work are:

1. A silver label generation method that combines semantic and lexical signals from label descriptions together with hierarchy-aware postprocessing.
2. A taxonomy-informed classifier that enhances label representations via graph propagation and uses them for multi-label prediction.
3. A semi-supervised training scheme that incorporates a limited LLM labeling budget, pseudo-label self-training,

and consistency regularization to improve robustness in the absence of gold labels.

## 2. PROBLEM DEFINITION

**Task Definition.** Our goal is to perform product review classification in a fully unlabeled setting. Formally, the input consists of an unlabeled text corpus $\mathcal{D}$ and a class taxonomy $\mathcal{T} = (\mathcal{C}, \mathcal{R})$, where $\mathcal{C}$ denotes the set of product classes and $\mathcal{R}$ represents directed parent–child relations between classes. Each directed edge $(c_i, c_j) \in \mathcal{R}$ indicates that class $c_j$is a sub-class of $c_i$, meaning the label space is organized as a hierarchical directed acyclic graph. The learning objective is to build a hierarchical multi-label text classifier $f(\cdot)$that maps a review document $d$ to a binary label vector $\mathrm{y} = [y_1, \ldots, y_{|\mathcal{C}|}]$, where $y_i = 1$ if $d$ belongs to class $c_i$ and $y_i = 0$ otherwise. In this task, each document is associated with at least two and at most three labels, which reflects the fact that a review may simultaneously describe multiple product aspects or categories while remaining relatively sparse in the label space.

**Data and Provided Resources.** The dataset confirms this unlabeled learning setup by providing two corpora of product reviews. A training corpus of 29,487 reviews and a test corpus of 19,658 reviews are both distributed without ground-truth class labels for direct supervision. Importantly, the task setting allows leveraging all provided information including the test corpus during training, which enables methods that benefit from a larger pool of unlabeled text. The label space contains 531 product categories and each category is identified by an index and a corresponding class name. To capture hierarchical structure, a separate taxonomy file is provided that lists parent–child relations between categories using their label indices, forming the class DAG used throughout the project. In addition, the dataset includes a class-related keyword resource. A set of keywords associated with the class name is provided for each class. These keywords offer additional semantic signals beyond the raw class names, which can be used to better represent each label and to support weak supervision strategies such as silver label generation.

**Constraints and Allowed Resources.** Since no manually labeled training set is available, model learning must rely on weak supervision and semi-supervised signals derived from the provided class information such as class names, hierarchy, and keywords and the unlabeled corpora. The task also allows using maximum 1,000 LLM (large language model) API calls for additional weak labeling under a limited budget. In our implementation, we used 250 LLM calls within the allowed budget to obtain high-precision label hints for a subset of unlabeled documents and incorporate them into the training process.

## 3. METHOD

### 3.1 Embedding Construction

#### *3.1.1 Pretrained BERT*

To construct dense representations without any labeled supervision, we first use bert-base-uncased as a frozen feature extractor. We load BertTokenizerFast and BertModel, move the model to GPU if available, set it to evaluation mode, and disable gradients so that the encoder parameters remain unchanged during embedding extraction. For each class, we build a short label description by concatenating the preprocessed class name and provided class-related keywords, which yields a natural-language label text. Both label texts and review documents are tokenized with padding and truncation up to 512 tokens and fed into BERT to obtain the final-layer contextual token embeddings. We then compute an embedding with fixed size via mean-pooling over tokens while masking out padding positions using the attention mask. Token embeddings are summed only over valid tokens and divided by the number of valid tokens to produce a single vector per text. Embeddings are generated in batches with batch size 64 for efficiency and saved as PyTorch tensors. Label embeddings are stored as a $(|\mathcal{C}|, d)$ matrix, while train/test document embeddings are stored along with sorted document.

#### *3.1.2 Domain-adaptive pretraining / Fine-tuning*

To better align the encoder with the domain-specific language in the provided product-review corpora, we additionally construct domain-adapted BERT embeddings via domain-adaptive pretraining (DAPT) before extracting document and label representations. Instead of using BertModel directly, we start from bert-base-uncased and perform masked language modeling (MLM) fine-tuning on the unlabeled texts available in the task. We build an MLM dataset from three sources: the unlabeled training reviews, the unlabeled test reviews (allowed to be used during training in this task), and the label texts constructed from class names and class-related keywords. Each text is tokenized with truncation and fixed-length padding up to a maximum length of 128 for efficient MLM updates. Therefore, batches are formed with data collator using mlm=True and masking probability 0.15. We then fine-tune BertForMaskedLM for 2 epochs with AdamW with learning rate $2 \times 10^{-5}$ and a linear learning-rate scheduler with warmup which is 10% of total steps. After training, the domain-adapted checkpoint is saved and reused later as the embedding backbone.

Once MLM fine-tuning is completed, the embedding extraction procedure follows the same structure as the non-fine-tuned baseline but now uses the DAPT-updated encoder. We reload the saved tokenizer and MLM model, take the underlying BERT encoder component, and run it in evaluation mode to compute contextual token embeddings. For both label texts and review documents, we apply the same mean-pooling strategy with attention masking to obtain a single fixed-size vector per text. Then we generate embeddings in batches with batch size 64 with truncation up to 512 tokens and save the resulting representations as PyTorch tensors used in the downstream pipeline for silver

label generation and hierarchy-aware classification. In summary, the key difference from the initial embedding construction is that the encoder is first adapted to the task domain via unsupervised MLM training on the provided corpora and the adapted encoder is subsequently used to re-encode both documents and labels into a more domain-specific embedding space.

## 3.2 Silver Label Generation

### *3.2.1 Similarity Scoring*

For silver label generation, we first compute a document–label relevance score by combining semantic and lexical similarity signals derived from label descriptions. Each label is represented as a short text by concatenating its class name and the provided class-related keywords. Using these label texts, we compute semantic similarity in an embedding space by taking the cosine similarity between document embeddings and label embeddings:

$$S_{emb}(d,c) = \frac{e_d^T e_c}{\|e_d\|\|e_c\|}$$

both sides are L2-normalized and the similarity matrix is obtained via a dot product. Then we compute lexical similarity by fitting a TF–IDF vectorizer on the label texts and transforming each document into the same TF–IDF space, and measuring cosine similarity between document vectors and label vectors. Per-document min–max normalization is applied to each similarity matrix, scaling scores within each document across all labels to $[0,1]$ to make the two similarity sources comparable. Finally, we fuse the normalized scores with a weighted sum:

$$S(d,c) = \alpha \tilde{S}_{emb}(d,c) + (1-\alpha)\tilde{S}_{lex}(d,c)$$

using a higher weight for semantic similarity and the remainder for lexical similarity. This yields a combined document–label score matrix that reflects both meaning-level alignment (embedding-based) and surface-level matching (TF–IDF), providing the basis for reliable downstream silver label assignment.

### *3.2.2 Silver Label Construction*

#### *3.2.2.1 Silver Label from score*

Given the combined document–label scores, we convert them into discrete silver labels using simple but taxonomy-aware heuristics that respect the task constraint of assigning 2–3 labels per document. First we rank all labels by their score and select a small set of high-scoring top-3 base candidates for each document. Among these candidates, labels whose scores fall below a minimum threshold are discarded to avoid injecting extremely weak matches. To incorporate hierarchical structure, we then expand the selected set by adding all ancestor labels of each chosen class using the provided parent–child DAG, which means repeatedly following child-to-parent links until reaching the root. This ancestor expansion encourages hierarchy-consistent label sets and reduces cases where a fine-grained label is predicted without its broader parent category. If the thresholding step results in an empty set, we fall back to the single best-scoring label and again add its ancestors to maintain a valid hierarchical assignment. After hierarchy expansion, we enforce the label-count constraint explicitly. Only the top-scoring ones up to the maximum of three are kept if more than three labels are collected and if fewer than two labels are obtained, additional high-scoring labels are added from the global ranking until the minimum of two labels is reached. The final output for each document is thus a compact label list that is both score-driven and hierarchy-aware.

#### *3.2.2.2 Multi-hot matrix for silver labels*

For efficient training, these per-document label lists are converted into a multi-hot label matrix $y_{\text{silver}} \in \mathbb{R}^{N \times C}$, where $N$ is the number of documents and $C = 531$ is the number of classes. Each row corresponds to a document and each column corresponds to a class. Entries are set to 1 for selected silver labels and 0 otherwise. This representation is standard for multi-label learning because it allows the classifier to be trained with vectorized loss functions such as binary cross-entropy over all classes and enables batching on GPU without custom per-sample label handling. In other words, the multi-hot matrix turns the heuristic silver assignments into a unified tensor format which can be directly consumed by the downstream model and training loop.

### *3.2.3 LLM-assisted Labeling*

After generating silver labels, we estimate their reliability using a simple confidence score derived from the combined similarity matrix. Confidence for each document is defined as the maximum document–label score across all classes, reflecting how strongly the document matches its best candidate label under our scoring function. Using a fixed threshold $t_{\text{silver}}$, we split the unlabeled corpus into a labeled set and an unlabeled set. Labeled set stands for documents with confidence above the threshold, whose silver labels are considered sufficiently reliable. In contrast, an unlabeled set means documents with lower confidence, where the silver labels are more uncertain and potentially noisy.

We further apply LLM-assisted labeling to a small subset of the lowest-confidence documents to improve label reliability under this fully unlabeled setting. Specifically, we select 250 documents from the unlabeled partition with the smallest confidence values and query an external LLM, Gemini API, to obtain higher-precision label suggestions. For controlling the LLM's output space we provide the model with a short candidate list. The top-$k$ labels from the similarity scores for each document are given, including for each candidate its label id, class name, and the constructed label description. The prompt instructs the LLM to choose the best 2–3 labels only from this candidate set or return an empty list if none are appropriate, and to output the result strictly as a JSON object. The returned labels are parsed and validated, ensuring ids are within the valid class range. Also, each result is saved as a JSONL record containing the document index and the selected label ids. These LLM-labeled samples serve as high-confidence anchors which will be incorporated in the subsequent semi-supervised training stage, where they are combined with silver labels to refine the labeled/unlabeled split and improve downstream learning.

## 3.3 Model and Prediction

### *3.3.1 Taxonomy-informed Classifier: Label GCN*

In hierarchical multi-label text classification, labels are strongly correlated through parent–child relations in the taxonomy. Leveraging this structure is especially helpful in our fully unlabeled setting, where supervision comes from noisy weak signals such as similarity-based silver labels. A taxonomy-informed classifier can share information across related labels. It encourages hierarchy-consistent predictions and improves robustness by propagating useful cues from well-supported labels to their neighbors in the label graph.

We construct a label graph directly from the provided taxonomy file and represent it with an adjacency matrix over the 531 classes. An edge between the two labels is added for each parent–child relation. We use an undirected version of the graph in implementation so that information can flow in both directions. In addition, self-connections are also included and a standard degree-based normalization is applied to obtain a fixed and scaled adjacency $\hat{A}$. It is then used as the message-passing operator in the GCN. This step is purely structural. It encodes which labels are related in the hierarchy and how strongly their representations should be mixed during propagation.

Given initial label embeddings from the embedding construction step, we apply a multi-layer LabelGCN to refine label representations using the taxonomy. Each GCN layer alternates between aggregating neighboring label embeddings via $\hat{A}$ and applying a learnable linear transformation with ReLU and dropout used in intermediate layers for regularization. In detail, starting from $H^{(0)}$, the initial label embedding matrix, each layer performs:

$$H^{(\ell+1)} = \hat{A}H^{(\ell)}W^{(\ell)}$$

where $W^{(\ell)}$ is a trainable weight matrix. Finally, the GCN-enhanced classifier scores each document by projecting its document embedding into the label embedding space and computing similarity-style logits against the GCN-refined label embeddings via a dot product. These logits are converted to multi-label probabilities with a sigmoid function and used for downstream training and prediction over the full 531-label space. Overall, this classifier explicitly integrates the taxonomy through LabelGCN while retaining a simple and efficient document–label matching mechanism, which makes it well-suited for large-scale hierarchical label spaces.

### *3.3.2 Prediction Method*

Using the taxonomy-informed classifier described in the previous section, our prediction pipeline operates entirely on precomputed embeddings and weak labels derived from the unlabeled corpora. First, each product review is represented by a fixed document embedding and each class is represented by an initial label embedding. These embeddings are the direct inputs to the GCN-enhanced classifier. Concretely, document embeddings are fed as $x$ while label embeddings initialize the trainable label table that is refined by LabelGCN over the normalized taxonomy adjacency. In forward inference, the model projects document embeddings into the label embedding space and produces per-class logits via a dot-product matching against the GCN-refined label embeddings. This design allows the model to score all 531 labels for any document using only embedding inputs, while injecting hierarchy information through graph propagation on the label side.

Then two weak labeling sources are combined to obtain supervision for training. We start with similarity-based silver labels and split documents by confidence, treating high-confidence silver labels as a provisional labeled set. We additionally incorporate LLM outputs saved from Gemini API calls for low-confidence documents. The LLM results are loaded from a JSONL file that stores {doc_index, labels} pairs and only entries with non-empty label lists are kept as valid LLM-labeled samples. These LLM labels are then converted into a multi-hot matrix $y_{\text{llm}}$, which is one row per LLM-labeled document, and appended to the existing labeled pool. The final labeled index set becomes the union of the high-confidence silver-labeled documents and the LLM-labeled documents. The unlabeled index set is updated by removing any documents that received LLM labels.

Lastly embedding-based datasets are prepared for downstream optimization. For low-confidence documents, we gather the corresponding document embeddings and their multi-hot targets by concatenating the silver label matrix rows and the LLM label matrix rows into a single label tensor. Then we construct a dataset for the unlabeled set, which provides only document embeddings without labels. It will later be used for semi-supervised components such as pseudo-labeling. After that, we split the labeled dataset into train/validation subsets with validation ratio 0.1 with a fixed random seed and build DataLoaders for labeled, validation, and unlabeled batches. After the model is trained in the next stage, predictions on any corpus are obtained by running the classifier on document embeddings to produce per-label probabilities and selecting 2–3 labels per document based on these scores using the same label-count constraint required by the task.

## 3.4 Semi-supervised Training

### *3.4.1 Overall Training Loop and Objectives*

We train the taxonomy-informed GCN-enhanced classifier on the embedding-based labeled set using a standard supervised multi-label objective. All document embeddings are treated as fixed inputs, and the model parameters, such as the document projection layer and the label-side GCN components including the trainable label embedding table, are optimized with AdamW on GPU when available. Since each instance can belong to multiple categories, binary cross-entropy with logits is used as the main loss function:

$$\mathcal{L}_{\text{sup}} = \sum_{c=1}^{C} \text{BCE}(y_c, \sigma(z_c))$$

which is applied independently to each of the 531 label dimensions. Training proceeds for up to 200 epochs with mini-

batches, and the labeled dataset is shuffled each epoch to stabilize optimization.

Model selection is performed using a held-out validation split created from the labeled pool. The current model on the validation loader is evaluated after each training epoch, tracking the sample-level F1 score at a fixed decision threshold. We maintain the best-performing checkpoint by storing the model state whenever validation F1 improves and also apply early stopping with a patience of five epochs to prevent overfitting and unnecessary computation. The final model used for downstream prediction is the checkpoint that achieves the highest validation F1 under this supervised training loop.

### *3.4.2 Self-training via Pseudo-label Generation*

For self-training, we iteratively expand the labeled supervision using pseudo-labels predicted on the unlabeled set. At regular intervals during training, in detail, every three epochs in our implementation, we run the current classifier in evaluation mode on the unlabeled loader to generate model-based labels. Logits for each unlabeled batch are computed and then converted into multi-label probabilities with a sigmoid function. Next, a high confidence threshold is applied to select pseudo-positive labels. For each sample, any label whose predicted probability exceeds the threshold is treated as a pseudo label. Samples that do not contain any label above the threshold are skipped to avoid injecting ambiguous or low-quality supervision. The remaining high-confidence unlabeled samples are collected into a pseudo dataset consisting of the original document embeddings paired with their predicted multi-hot pseudo-label vectors.

Once generated, the pseudo dataset is integrated into the training loop by concatenating it with the original labeled training set, forming an updated training dataset for the next epochs. This implements a conservative self-training scheme. Only predictions the model is strongly confident about are fed back as additional training targets while uncertain samples remain in the unlabeled pool. The motivation is twofold. First, pseudo-labeling increases the effective amount of supervision without exceeding the task's unlabeled constraint, which allows the classifier to learn from a broader coverage of the data distribution. Second, using a strict threshold helps maintain label reliability by minimizing error reinforcement. As the model improves, more unlabeled samples cross the confidence threshold. It gradually enlarges the pseudo-labeled set and sharpening decision boundaries. This progressive expansion of training signals improves learning effectiveness by leveraging the unlabeled corpus to provide additional, model-consistent supervision beyond the initial silver/LLM-labeled anchors.

### *3.4.3 Consistency Regularization*

For further stabilizing training under noisy weak supervision, consistency regularization is applied by encouraging the model to produce similar predictions for a document embedding under small perturbations. For each labeled batch an augmented embedding $X_{\text{aug}}$ is created by adding Gaussian noise to the original embedding $X$. We then run the classifier twice, once on $X$ and once on $X_{\text{aug}}$, and compute the standard supervised loss using the logits from the clean input. In parallel, a consistency loss is computed:

$$\mathcal{L}_{\text{cons}} = \|\sigma(f(x)) - \sigma(f(x+\epsilon))\|^2$$

by taking sigmoid probabilities from both forward passes and minimizing their mean-squared error. It penalizes prediction drift caused by the perturbation. The final training objective is a weighted sum of the supervised loss and the consistency term with $\lambda_{\text{cons}} = 0.1$:

$$\mathcal{L} = \mathcal{L}_{\text{sup}} + \lambda\, \mathcal{L}_{\text{cons}}$$

This regularizer improves learning effectiveness by smoothing the decision function in the embedding space. It reduces sensitivity to small input changes, mitigates overfitting to imperfect silver/pseudo labels, and also encourages more stable and calibrated predictions which generalize better to unseen reviews.

# 4. EXPERIMENTAL RESULTS

## 4.1 Evaluation Metrics

The proposed approach is evaluated using four standard metrics for multi-label classification: accuracy, sample-wise F1 (F1-samples), micro-averaged F1 (F1-micro), and macro-averaged F1 (F1-macro). Let $y_{i,c} \in \{0,1\}$ and $\hat{y}_{i,c} \in \{0,1\}$ denote the ground-truth and predicted labels for instance $i$ and class $c$, where predictions are obtained by thresholding sigmoid probabilities. **Accuracy** is computed in a multi-label (subset) sense by comparing predicted and true label vectors and is reported as the fraction of instances that are correctly predicted. **Micro-F1** aggregates true positives, false positives, and false negatives over all labels before computing F1:

$$F1_{\text{micro}} = \frac{2\,TP}{2\,TP + FP + FN}$$

which emphasizes overall performance dominated by frequent labels. **Macro-F1** computes F1 independently for each label and then averages across labels:

$$F1_{\text{macro}} = \frac{1}{C}\sum_{c=1}^{C} F1_c$$

which treats rare and frequent classes equally and, being sensitive to performance on tail labels. Finally, **F1-samples** computes an F1 score for each instance and averages across instances:

$$F1_{\text{samples}} = \frac{1}{N}\sum_{i=1}^{N} \frac{2\,|\,Y_i \cap \hat{Y}_i\,|}{|\,Y_i\,| + |\,\hat{Y}_i\,|}$$

where $Y_i$ and $\hat{Y}_i$ are the true and predicted label sets for sample $i$.

Among these, we use F1-samples as the primary selection criterion during training such as validation monitoring and early stopping and as the main score to compare successive system variants. This choice aligns well with our task setting, where each document is associated with a small set of labels from a large taxonomy of 531 classes. Instance-level correctness, recovering the right label set per document, matters more directly than optimizing for globally frequent labels that micro-F1 tends to emphasize. Macro-F1 can be overly volatile due to many low-support classes in an unlabeled/weakly supervised setup. F1-

**Table 1. Incremental performance gains by adding components**

| Stage | Key Change (added on top of previous) | Silver labels | GCN classifier | Self-training | Consistency regularization | DAPT Fine-tuning | LLM Labels used | F1-samples |
|---|---|---|---|---|---|---|---|---|
| S1 | Baseline (Silver labels + GCN classifier) | O | O | X | X | X | X | 0.12819 |
| S2 | + SSL training (Self-training, Consistency regularization) | O | O | O | O | X | X | 0.22834 |
| S3 | SSL with hyperparameter tuned | O | O | O | O | X | X | 0.30910 |
| S4 | + DAPT (MLM Fine-tuned BERT embeddings) | O | O | O | O | O | X | 0.33415 |
| S5 | + LLM-assisted labeling (Gemini) | O | O | O | O | O | O | 0.36662 |

samples provides a balanced instance-centric view by rewarding overlap between predicted and true label sets on each document, which makes it a practical and stable criterion for tracking improvements across our incremental pipeline stages.

## 4.2 Main Results

Table 1 summarizes the incremental improvements obtained by progressively adding components to the pipeline, where F1-samples is used as the primary metric. Starting from the baseline that uses only silver-label supervision with the GCN-enhanced classifier (S1, 0.12819), introducing semi-supervised training with self-training and consistency regularization yields a large performance gain (S2, 0.22834). After tuning the SSL hyperparameters, the score further increases to 0.30910 (S3). Replacing the original embeddings with DAPT (MLM) fine-tuned BERT embeddings provides an additional improvement (S4, 0.33415). Finally, incorporating a small set of LLM-assisted labels (Gemini API) as high-quality anchors achieves the best result in our experiments (S5, 0.36662). It demonstrates that both stronger in-domain representations and limited but reliable external supervision can complement the silver-label based training effectively.

## 4.3 Hyperparameter sensitivity

**Table 2. Hyperparameter sensitivity during S2**

| Variant | $t_{silver}$ | $t_{pseudo}$ | $\lambda_{cons}$ | F1-samples |
|---|---|---|---|---|
| V1 | 0.8 | 0.9 | 0.1 | 0.22834 |
| V2 | 0.7 | 0.9 | 0.1 | 0.28154 |
| V3 | 0.7 | 0.85 | 0.1 | 0.30910 |
| V4 | 0.7 | 0.8 | 0.1 | 0.28057 |
| V5 | 0.7 | 0.85 | 0.2 | 0.28350 |
| V6 | 0.7 | 0.85 | 0.05 | 0.28879 |

Table 2 reports the hyperparameter sensitivity analysis conducted after introducing SSL training. Three key thresholds and weights are tuned: the silver-label confidence threshold $t_{\text{silver}}$ used for the labeled/unlabeled split, the pseudo-label selection threshold $t_{\text{pseudo}}$, and the consistency loss weight $\lambda_{\text{cons}}$. Starting from the initial setting $(t_{\text{silver}}, t_{\text{pseudo}}, \lambda_{\text{cons}}) = (0.8, 0.9, 0.1)$ (V1, F1-samples = 0.22834), we first fixed $t_{\text{pseudo}} = 0.9$ and $\lambda_{\text{cons}} = 0.1$ and varied only $t_{\text{silver}}$. Lowering $t_{\text{silver}}$ from 0.8 to 0.7 improved F1-samples to 0.28154 (V2), indicating that a slightly more permissive split can provide a stronger supervised signal without overly degrading label quality. Next, with $t_{\text{silver}} = 0.7$ fixed, we varied $t_{\text{pseudo}}$ across $\{0.9, 0.85, 0.8\}$. The best performance was obtained at $t_{\text{pseudo}} = 0.85$ (V3, 0.30910), while both a stricter threshold (V2) and a looser threshold (V4) resulted in lower scores. It suggests a trade-off between pseudo-label coverage and noise. Finally, keeping $(t_{\text{silver}}, t_{\text{pseudo}}) = (0.7, 0.85)$ fixed, we adjusted $\lambda_{\text{cons}}$ from 0.1 to 0.2 (V5) and 0.05 (V6). Neither change improved performance over $\lambda_{\text{cons}} = 0.1$. Therefore, the configuration (0.7, 0.85, 0.1) provided the best balance among the three hyperparameters and was used in subsequent experiments.

## 4.4 LLM Supervision budget

**Table 3. LLM Supervision budget during S5**

| Variant | LLM API calls | Valid labels | F1-samples |
|---|---|---|---|
| V1 | 1000 | 750 | 0.34161 |
| V2 | 250 | 185 | 0.36662 |
| V3 | 500 | 372 | 0.34565 |

Table 3 summarizes the effect of the LLM supervision budget in stage S5 by varying the number of LLM API calls. We additionally report the number of valid labels, defined as LLM outputs that are not an empty list. That is, the LLM returned at least one label ID for valid labels. Among the tested budgets, using 250 API calls produced 185 valid labeled instances and achieved the best performance (F1-samples = 0.36662), outperforming both larger-call settings such as 500 and 1000 calls in our experiments.

# 5. DISCUSSION

## 5.1 Analysis of incremental improvements

The incremental results in Table 1 show that most of the performance gains come from reducing label noise while gradually expanding the amount of usable supervision. The baseline (S1) relies solely on similarity-based silver labels together with the taxonomy-informed GCN classifier. However, it is still bottlenecked by the inherent noise and sparsity of silver supervision in a 531-way hierarchical label space. As a result, the model has limited opportunity to learn reliable decision boundaries, especially for fine-grained leaf categories and the overall F1-samples remains low (0.12819).

A major improvement is observed when semi-supervised training is introduced (S2). Self-training allows the model to turn

a portion of unlabeled instances into additional training signal and consistency regularization reduces the risk of overfitting to noisy supervision by encouraging stable predictions under small perturbations. This combination yields a large gain from 0.12819 to 0.22834. It suggests that the initial classifier trained on silver labels is already informative enough to bootstrap additional pseudo-labeled data and that regularization is important to prevent error amplification during bootstrapping. The hyperparameter sweep in Table 2 further supports this interpretation. Lowering the silver-confidence threshold from $t_{\text{silver}} = 0.8$ to $0.7$ increases performance ($0.22834 \rightarrow 0.28154$), implying that a slightly larger labeled pool can be beneficial when combined with SSL mechanisms that are robust to moderate label noise. Fixing $t_{\text{silver}} = 0.7$, the best pseudo-label threshold is $t_{\text{pseudo}} = 0.85$ (0.30910), whereas a stricter threshold (0.9) likely produces too few pseudo labels to meaningfully expand coverage, and a looser threshold (0.8) likely injects too much noise. Finally, the consistency weight $\lambda_{\text{cons}}$ shows a clear optimum at 0.1. Increasing it (0.2) or decreasing it (0.05) both degrade performance, indicating that the regularization term should be strong enough to stabilize training but not so strong that it suppresses learning from the supervised signal. Overall, the tuned SSL configuration (S3) achieves a substantial additional gain to 0.30910, demonstrating that effective bootstrapping in this task requires careful balancing between coverage and noise control.

Replacing the original embeddings with DAPT (MLM) fine-tuned BERT embeddings yields another consistent improvement (S4, $0.30910 \rightarrow 0.33415$). This gain is expected because the pipeline depends on embedding quality at multiple points. Higher-quality in-domain representations improve the separation between semantically similar categories, reduce ambiguity in document–label similarity scoring, thereby improving silver label quality, and provide a stronger input space for the GCN-enhanced classifier to learn from. In other words, DAPT strengthens both the supervision signal and the model's representation capacity, which means better silver labels and better document embeddings. It collectively improves downstream classification performance.

Finally, adding a small amount of LLM-assisted supervision produces the best result (S5, 0.36662). Although the LLM budget is limited, the added labels act as high-precision anchors that can correct particularly uncertain or noisy regions of the label space and reduce the reliance on purely similarity-derived silver labels. Table 3 indicates that more LLM calls do not monotonically translate to better performance. The best variant uses 250 calls with 185 valid labeled-instances and outperforms 500 and 1000 calls settings. This suggests that the quality and selection of LLM-labeled samples can matter more than raw quantity, especially in a hierarchical multi-label setting where incorrect labels can propagate noise into training. Taken together, Tables 1–3 show that the strongest gains come from introducing SSL with carefully tuned thresholds to expand supervision while controlling noise, improving representation quality through domain-adaptive MLM fine-tuning, and injecting a small but reliable set of LLM-derived labels to further stabilize and refine learning in the most ambiguous cases.

## 5.2 Limitations

A key limitation is the noise in silver labels. Similarity-based supervision is only an approximation, so some labels are missing or incorrect, especially for short or ambiguous reviews and fine-grained leaf categories. This weak supervision can introduce early bias and, even with thresholds, some noise can still propagate into later training stages. Another limitation is hierarchy-induced bias. Enforcing taxonomy consistency such as adding ancestors and propagating information on the label graph improves structure-aware learning. However, it can also encourage overly generic predictions or spread errors along correlated branches. The use of LLM supervision is sparse and potentially biased by selection. Only a small subset can be labeled and the benefit depends heavily on which samples are chosen. Moreover, restricting the LLM to a candidate label shortlist can inherit mistakes from the retrieval/scoring step. Finally, the pipeline is largely embedding-based rather than end-to-end, which is efficient but limits the model's ability to adapt representations specifically for downstream classification. Also, it may miss subtle, context-dependent cues that require token-level modeling.

## 5.3 Case study: one success and one failure

A useful case study from our experiments is the impact of the LLM supervision budget, which highlights that 'more external supervision does not necessarily yield better performance' in a weakly-supervised hierarchical multi-label setting. As a successful example, using a small but targeted amount of LLM labeling, 250 API calls with 185 valid labels, produced the best result. This suggests that a limited number of high-quality LLM labels can act as reliable anchors that correct silver-label noise and stabilize semi-supervised training, which improves generalization even when the overall labeled set remains small.

In contrast, a failure case is that increasing the budget to a much larger number of LLM calls did not improve performance. This indicates that the benefit of LLM supervision is quality-dependent and selection-dependent, rather than purely quantity-driven. A plausible interpretation is that as more samples are sent to LLM, the pool increasingly includes harder and more ambiguous documents and candidate-label lists where the correct label may be missing. Consequently, the additional LLM outputs can introduce noisier or less consistent supervision that dilutes the gains from the initial clean labels. Thus, this case study suggests that the LLM is most useful for adding a small number of reliable correction labels for uncertain samples and that simply increasing the number of LLM calls does not guarantee better performance.

# 6. CONCLUSION

We addressed fully unlabeled hierarchical multi-label text classification by building a weakly- and semi-supervised pipeline that leverages label taxonomy, label text resources, and the unlabeled review corpora. Starting from pretrained BERT embeddings, we improved representation quality through domain-adaptive pretraining (MLM-based DAPT) and used the resulting document/label embeddings to generate silver labels via a fusion

of semantic and lexical similarity. Then a taxonomy-informed classifieris adopted to model label dependencies explicitly, which refines label representations with LabelGCN and predicts labels through document–label matching in the learned embedding space. We further strengthened learning with semi-supervised training, combining high-confidence silver supervision with self-training and consistency regularization and incorporated a limited amount of LLM-assisted labeling to provide reliable anchors for uncertain samples.

Experimental results showed that each component contributed to the final performance. Introducing SSL produced the largest jump over the baseline and careful threshold tuning improved stability. DAPT further enhanced results by aligning embeddings to the domain and a small set of LLM-derived labels yielded the best overall score. These findings highlight that effective learning depends on balancing supervision coverage and reliability while exploiting taxonomy structure to share signal across related classes in a large hierarchical label space.

Future work could improve the pipeline in several directions. First, candidate label generation for both silver labeling and LLM prompting could be strengthened, such as better retrieval or hierarchy-aware candidate expansion, to reduce upstream bias. Second, more principled hierarchy-aware objectives and calibration strategies could help mitigate parent-label overprediction and improve fine-grained leaf discrimination. Finally, moving from fixed embeddings to a lightweight end-to-end approach, while still respecting the unlabeled constraint, may allow the encoder to adapt more directly to the downstream hierarchical classification objective and further improve generalization.

## 7. REFERENCES


[1] Shen, J., Qiu, W., Meng, Y., Shang, J., Ren, X., and Han, J. 2021. TaxoClass: Hierarchical Multi-Label Text Classification Using Only Class Names. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT).* Association for Computational Linguistics.

[2] Ayash, L., Algarni, A., and Alqahtani, O. 2025. Label-Aware Hierarchical Ranking Model for Multi-Label Text Classification. *IEEE Access* 13 (2025).

[3] Gao, Y., Zheng, X., and Sun, H. 2025. HiGATA: Hierarchy-aware Graph Attention Networks Enhanced by Topological Analysis for Hierarchical Multi-Label Text Classification. In *Proceedings of the International Joint Conference on Neural Networks (IJCNN).* IEEE.

[4] Wu, D., Wu, H., and Li, J. 2025. Hierarchy-Aware Adaptive Graph Neural Network. *IEEE Transactions on Knowledge and Data Engineering* 37, 1 (2025).